

Question 1
SIMATS ENGINEERING
ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)*

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I _Shaik Ashraf/192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Question 1

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis

Question 1

Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented
---------	-----	---------------------------------	-----------------	------------------	------------------

Question 1
Question 1 — Big Data Platform for Digital Customer Activity

A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Answer

The volume and variety described strongly suggest a distributed cloud or Hadoop-style architecture rather than a single database. A likely design separates inexpensive raw storage from processing and serving layers. The platform would use a data lake for raw events, distributed processing for historical jobs, a streaming layer for immediate events, and a curated warehouse or serving database for dashboards.

Probable storage and ingestion design

Website clicks, mobile interactions and transactions arrive in different formats. Transaction data is mostly structured, while clickstream and application events are commonly semi-structured JSON. A distributed object store such as Amazon S3 or Azure Data Lake Storage, or an HDFS-style system, is a likely raw layer. Kafka or a managed equivalent would ingest continuous events. Batch connectors and ETL jobs would bring periodic files or database changes into the same lake.

- Structured transaction tables would be kept as curated relational or columnar datasets, while semi-structured events could retain flexible JSON structure at ingestion.
- Data would be partitioned by date, region, source or customer segment so queries scan only the required data.
- Columnar formats such as Parquet would reduce I/O for analytical queries.
- Kafka topics could be partitioned by event type or region so multiple consumers process events in parallel.

Processing, indexing and business intelligence

Apache Spark is a strong candidate for batch processing because it distributes transformations across many machines. Spark Structured Streaming or Apache Flink can handle near-real-time events. Cleaned data would move to a cloud warehouse or SQL serving layer, and Power BI or Tableau would normally connect to curated datasets rather than raw event files.

- Partition pruning, clustering or indexes would improve queries on date, customer, region and transaction type.
- Horizontal scaling would allow worker nodes to be added when event volume increases.
- Dashboards would read pre-aggregated metrics such as sales, conversion rate, active users and customer segments.
- Monitoring would track ingestion lag, failed jobs, partition imbalance and query latency.

Question 1

Table 1 — Reverse-Engineered Architecture

Layer	Likely component	Reverse-engineered purpose
Event ingestion	Kafka / Kinesis	Collect and distribute high-volume events
Raw storage	S3 / ADLS / HDFS	Retain scalable raw data
Batch processing	Apache Spark	Clean, join and aggregate history
Streaming	Flink / Spark Streaming	Near-real-time analytics
Analytical format	Parquet / columnar tables	Efficient analytical scans
Serving	Cloud warehouse / SQL layer	Expose curated business data
BI	Power BI / Tableau	Dashboards and KPI reporting

Result

The most probable design is a layered distributed platform: events enter through a messaging system, are retained in distributed storage, processed with Spark or a similar engine, and exposed through a curated analytical layer. Partitioning and horizontal scaling explain how billions of events can be handled without relying on one machine. The BI layer is separated from raw storage so dashboards remain responsive and business users work with consistent metrics.

Question 1
Question 1 — Flowchart



Question 2

Question 2 — E-Commerce Personalization Pipeline

An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Answer

The platform needs both immediate customer context and historical behavioural analysis. A likely design combines an event-streaming backbone, a low-latency customer profile store, a cache for frequently requested recommendations, and a distributed analytical store. The recommendation path must respond quickly while learning from previous sessions and purchases.

Event streaming and customer profile

Cart events, product views, checkout attempts and payments can be published to Kafka or a managed equivalent. Flink or Spark Structured Streaming can join recent events with customer profiles and maintain session-level features. Redis, Cassandra or DynamoDB are likely for frequently accessed customer and session information. Redis can also cache product recommendations and popular results.

- Session IDs connect page views, cart actions and checkout events so the current customer journey can be reconstructed.
- Structured orders can be joined with semi-structured clickstream data through customer ID, session ID, product ID and timestamps.
- Streaming updates short-term features such as recently viewed products, cart value and abandonment status.
- Replication, checkpointing and replayable event logs provide fault tolerance.

Recommendation and analytics workflow

The recommendation process would normally collect events, build customer and product features, generate candidates, rank them with a machine-learning model, and cache the final results. Historical data can be processed with Spark or Trino/Presto. Dashboards can monitor conversion, abandonment, revenue and recommendation performance.

- Collaborative filtering can learn from purchase and browsing behaviour, while content-based features help with new products.
- Feature pipelines can combine recent session behaviour with longer-term preferences.
- Distributed query engines support interactive analysis across large clickstream and order datasets.
- Regional partitioning and replication can reduce latency and isolate failures.
- Dashboards can track conversion rate, cart abandonment, average order value, recommendation click-through and regional revenue.

Question 2

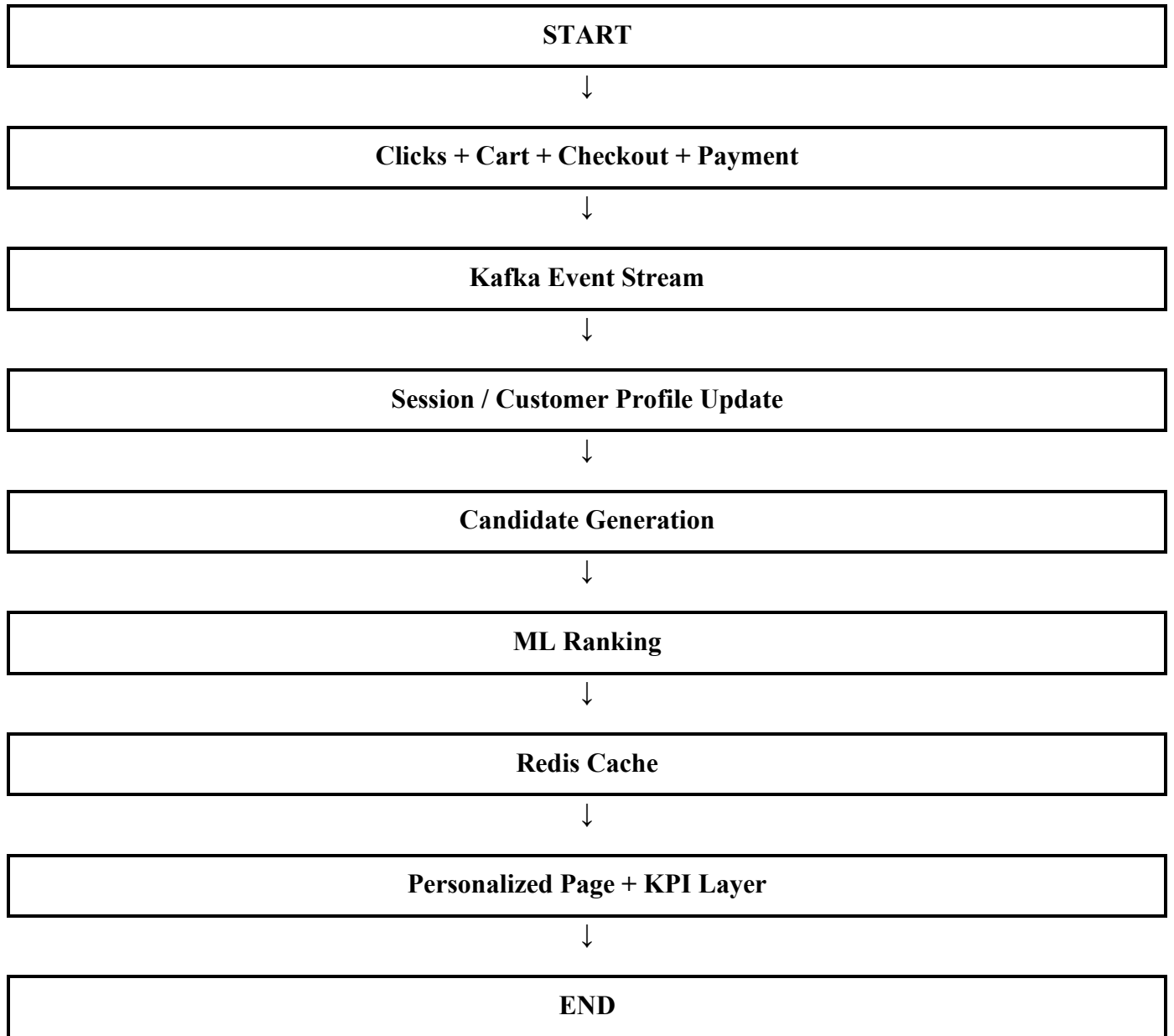
Table 2 — Reverse-Engineered Architecture

Component	Likely technology	Reason for inference
Event stream	Kafka / Kinesis	Continuous customer events
Profile store	Redis / Cassandra / DynamoDB	Fast customer and session lookup
Processing	Flink / Spark Streaming	Real-time feature updates
Cache	Redis	Low-latency recommendations
Query layer	Trino / Presto / Spark SQL	Distributed analytics
ML	Collaborative filtering + ranking	Personalization
BI	Power BI / Tableau	KPI and funnel monitoring

Result

The probable architecture is a real-time personalization loop: customer events enter a durable stream, recent behaviour updates a profile and feature layer, a recommendation model ranks products, and the result is cached for fast delivery. Historical data is processed for deeper analysis and retraining. Replication, replayable streams and distributed processing provide the scalability and fault tolerance expected from a multi-region platform.

Question 2
Question 2 — Flowchart



Question 3
Question 3 — Healthcare Big Data and Clinical Analytics

A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Answer

The mixture of patient records, images, sensor streams and prescriptions points to a hybrid architecture rather than one database. Structured clinical and billing information needs transactional consistency, images require object or specialized storage, and wearable data adds a continuous stream. Because healthcare data is sensitive, security and auditability would be built into every layer.

Hybrid storage and integration

Patient demographics, encounters, prescriptions and insurance records are likely stored in relational databases or a clinical warehouse. Diagnostic images can be retained in object storage or an imaging repository, while wearable readings can flow through Kafka or a cloud messaging service. Healthcare interoperability standards can connect hospitals, laboratories and insurers. A common patient identity layer is important for joining information safely.

- Structured records can be normalized in relational tables, while images and documents are stored as objects with searchable metadata.
- FHIR-based APIs and integration mechanisms can support controlled exchange between systems.
- Streaming pipelines can calculate current wearable indicators without putting every raw event into an operational database.
- Data lineage should identify source, transformation history and permitted use.

Analytics, privacy and machine learning

Historical data can be processed with Spark for cohort studies, disease-risk analysis and population health research. Streaming can detect abnormal wearable patterns and create alerts. Machine-learning models may use history, laboratory values, medications and sensor features to estimate risk, but clinical decisions should remain subject to validation and professional oversight.

- Encryption should protect data in transit and at rest.
- Role-based or attribute-based access should restrict information according to job responsibility.
- Audit logs should record access to sensitive records.
- De-identification or pseudonymization can support research when direct identity is unnecessary.
- Model monitoring is needed to detect drift, bias and unsafe changes in prediction quality.

Question 3

Table 3 — Reverse-Engineered Architecture

Data type	Likely storage / tool	Main use
Patient records	Relational DB / clinical warehouse	Structured clinical history
Medical images	Object / imaging repository	Diagnostic image storage
Wearables	Kafka / IoT + time-series store	Real-time monitoring
Integration	FHIR APIs / integration engine	Cross-organization exchange
Analytics	Spark + distributed SQL	Population studies
ML	Python ML / Spark ML	Risk prediction
Security	IAM + encryption + audit	Privacy and compliance

Result

The reverse-engineered system is most likely a hybrid healthcare platform with transactional databases, object storage, streaming infrastructure and a governed analytical layer. The technical architecture must be inseparable from privacy controls because patient information can cause serious harm if exposed or misused. Analytics can support clinical decision-making and population studies, but models should be validated, monitored and used within clinical governance.

Question 3
Question 3 — Flowchart



Question 4

Question 4 — Smart City Traffic and Urban Analytics

A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

Answer

The continuous feeds suggest an edge-to-cloud architecture. Some filtering and immediate detection should happen near the data source to reduce network load and latency, while a central platform retains information for city-wide analysis. Geospatial and time-series databases are especially suitable because location and time are central to traffic analytics.

Edge, messaging and storage architecture

Roadside devices and gateways can perform filtering, aggregation and anomaly detection before sending events to Kafka or MQTT brokers. Centralized storage can retain historical sensor data and CCTV metadata. TimescaleDB or InfluxDB can support time-based queries, while PostGIS-enabled PostgreSQL can handle location-aware operations.

- Edge processing reduces bandwidth by filtering duplicate or low-value events.
- Kafka partitions can separate regions, roads or event types and let multiple consumers work independently.
- Time-series storage supports speed and vehicle-count queries, while spatial databases support road and area analysis.
- Historical data can be retained in a data lake for long-term planning and model training.

Streaming, prediction and visualization

Flink or Spark Structured Streaming can combine traffic speed, vehicle counts, weather and transport activity to calculate congestion indicators. Machine-learning models can predict congestion from current and historical conditions. Batch Spark jobs can examine long-term patterns. Dashboards can combine live maps, alerts and historical trends for traffic-control teams.

- Geospatial joins can connect sensor events to road segments and transport routes.
- Real-time alerts can identify abnormal congestion or sudden drops in average speed.
- Access control should separate public information from sensitive operational data.
- Encryption, device authentication and network segmentation can protect infrastructure.
- Dashboard users should receive role-appropriate views, with audit logs for sensitive access.

Question 4

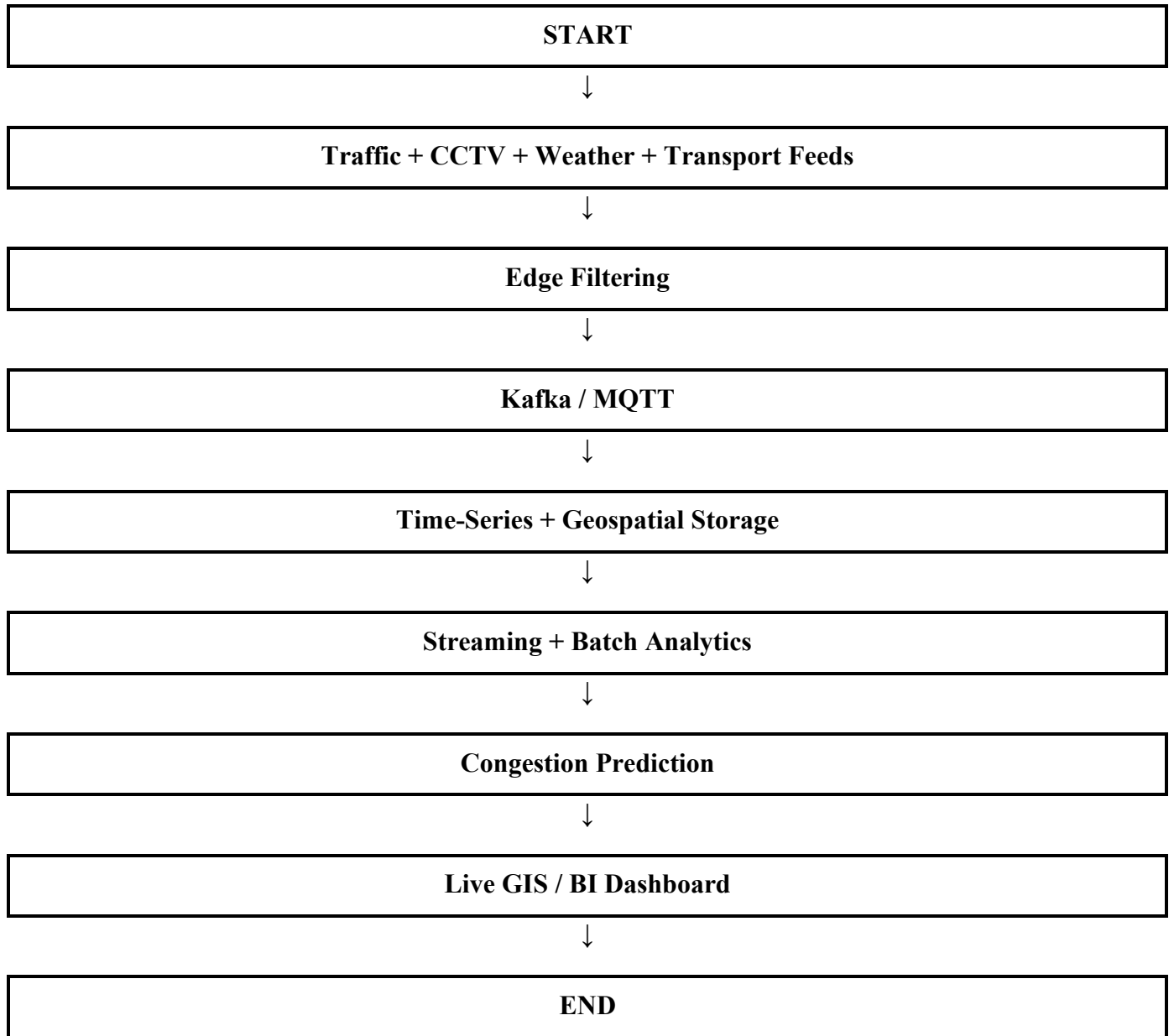
Table 4 — Reverse-Engineered Architecture

Layer	Likely technology	Purpose
Edge	IoT gateway / edge device	Local filtering and fast response
Messaging	Kafka / MQTT	Reliable event transport
Time series	TimescaleDB / InfluxDB	Time-based sensor queries
Geospatial	PostGIS	Location-aware analysis
Processing	Flink / Spark Streaming	Live congestion computation
Historical analytics	Spark + data lake	Urban planning analysis
Visualization	Power BI / GIS dashboard	Live operational intelligence
Security	IAM, encryption, segmentation	Protect citizen and infrastructure data

Result

The likely smart-city platform is an edge-to-cloud pipeline. Edge devices reduce latency and bandwidth use, distributed messaging provides reliable ingestion, and specialized databases make time and location queries efficient. Streaming supports immediate traffic operations, while batch analytics reveals long-term patterns. Visualization then turns results into maps, alerts and planning indicators.

Question 4
Question 4 — Flowchart



Question 5 — Healthcare Analytics and Predictive Health

A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Answer

This system is a governed healthcare analytics platform with emphasis on longitudinal patient data. The likely design separates operational clinical records from large analytical datasets while keeping a controlled integration layer. Wearable readings add streaming data, while laboratory and appointment information can enter through scheduled or event-driven pipelines.

Hybrid storage and ETL

Relational databases are likely for patient, appointment and laboratory records. Reports and documents can be stored as objects with metadata, while wearable readings can be kept in a time-series store or streamed through Kafka. ETL/ELT pipelines can standardize identifiers, validate units, remove duplicates and create a common patient timeline. FHIR-based interoperability can support controlled exchange.

- Patient identity matching is critical because the same person may appear in multiple systems.
- ETL should validate quality and record source and transformation history.
- Streaming can update current health indicators without waiting for daily batch loading.
- Distributed storage and columnar formats make large longitudinal datasets more efficient to query.

Predictive analytics, governance and clinical use

Spark or another distributed framework can analyse large patient datasets for population trends, readmission patterns and risk factors. Real-time processing can monitor wearable readings and create alerts. Machine-learning models may produce diagnosis-risk scores or treatment-support recommendations, but outputs should be explainable, validated and reviewed by qualified professionals before high-impact use.

- Encryption at rest and in transit protects sensitive health information.
- Role-based access, least privilege and multi-factor authentication reduce unauthorized access.
- Audit logs capture access, modification and data-sharing events.
- De-identification can support population research where identity is unnecessary.
- Model governance should cover validation, bias testing, performance monitoring and controlled retraining.

Table 5 — Reverse-Engineered Architecture

Area	Likely approach	Purpose
Operational records	Relational clinical databases	Reliable structured storage
Documents	Object storage + metadata	Unstructured medical information
Wearables	Kafka + time-series store	Real-time monitoring
ETL / interoperability	FHIR APIs + ETL/ELT	Integrate hospitals and clinics
Distributed analytics	Spark / distributed SQL	Population-level analysis
ML	Risk scoring + predictive models	Decision support
Governance	IAM + encryption + audit	Privacy and compliance

Result

The reverse-engineered solution is a governed hybrid analytics platform in which operational healthcare systems feed a standardized analytical environment. Distributed processing makes population-scale analysis practical, while streaming supports current patient monitoring. Security and compliance mechanisms surround the entire pipeline, and predictive models should be treated as decision-support tools with validation, monitoring and professional oversight.

Question 4

Question 5 — Flowchart

